What is a Kubernetes CronJob?

A **CronJob** in Kubernetes is used to **run Jobs on a time-based schedule**, similar to Linux crontab.

CronJob = Scheduled Job

Examples:

- Nightly backups
- Log cleanup
- Report generation
- DB maintenance
- Periodic sync tasks

Why CronJob?

❌ **Problem without CronJob**

- Manual job execution
- External schedulers
- No Kubernetes-native retry & status tracking

✅ **CronJob benefits**

- Native Kubernetes scheduling
- Retry & failure handling (via Job)
- Centralized observability
- Works with RBAC, secrets, volumes

CronJob Architecture & Internals (IMPORTANT)

Controllers involved:

1. **CronJob Controller**
2. **Job Controller**
3. **kubelet**

Internal Workflow

1. CronJob controller checks schedule
2. When time matches:
    - Creates a **Job**
3. Job controller:

    - Creates **Pod(s)**
4. Pod runs task
5. Job completes (Success / Failed)
6. CronJob updates history

CronJob never runs Pods directly

🔹 **Basic CronJob Example**

```
apiVersion: batch/v1
kind: CronJob
metadata:
 name: hello-cron
spec:
 schedule: "*/5 * * * *"
 jobTemplate:
  spec:
   template:
    spec:
     containers:
     - name: hello
      image: busybox
```

```
          command: ["sh", "-c", "date; echo Hello from CronJob"]
        restartPolicy: OnFailure
```

Apply:

```
kubectl apply -f cronjob.yaml
```

Check:

```
kubectl get cronjobs
kubectl get jobs
kubectl get pods
```

Cron Schedule Format (VERY IMPORTANT)

```
* * * * *
| | | | |
| | | | +-- Day of week (0–6)
| | | +---- Month (1–12)
| | +------ Day of month (1–31)
| +-------- Hour (0–23)
+---------- Minute (0–59)
```

Example:

- 0 2 * * * → Daily at 2 AM
- */10 * * * * → Every 10 mins

🔹 **Timezone Handling (TRICKY)**

By default:

- CronJob uses **kube-controller-manager timezone**
- Usually **UTC**

Kubernetes ≥1.27

```
spec:
  timeZone: Asia/Kolkata
```

🔹 **concurrencyPolicy (CRITICAL)**

Controls overlapping executions.

concurrencyPolicy: Allow | Forbid | Replace

Policy	Behavior
Allow (default)	Jobs run in parallel
Forbid	Skip next run if previous still running
Replace	Kill running job and start new

🔹 **Job History Limits (Cleanup)**

successfulJobsHistoryLimit: 3

failedJobsHistoryLimit: 1

✔ Prevents cluster clutter

🔹 **Starting Deadline (Missed Schedules)**

startingDeadlineSeconds: 200

If schedule missed due to:

- Controller restart
- API server downtime

Job will start **only if within deadline**

🔹 **Suspending a CronJob**

spec:

suspend: true

Use case:

- Maintenance window
- Incident freeze

🔹 **CronJob Retry & Failure Handling**

Handled by **Job spec**:

```
jobTemplate:
  spec:
    backoffLimit: 3
    activeDeadlineSeconds: 120
```

🔹 **CronJob with ConfigMap & Secret**

```
envFrom:
- configMapRef:
    name: app-config
- secretRef:
    name: db-secret
```

🔹 **Real-World Production Examples**

---

1 **Database Backup CronJob**

```
schedule: "0 1 * * *"
concurrencyPolicy: Forbid
```

- Runs daily at 1 AM
- Prevents overlapping backups
- Pushes to S3/GCS

2 Log Cleanup Job

schedule: "0 */6 * * *"

successfulJobsHistoryLimit: 1

- Deletes old logs
- Keeps minimal history

---

3 Data Sync Job

parallelism: 5

completionMode: Indexed

- Syncs sharded data

3 Data Sync Job

parallelism: 5

completionMode: Indexed

- Syncs sharded data

🔹 **Debugging CronJobs (PRODUCTION)**

Describe CronJob

kubectl describe cronjob <name>

Check Jobs created

kubectl get jobs --watch

Logs

kubectl logs job/<job-name>

Logs

kubectl logs job/<job-name>

🔹 **Common Issues & Fixes**

Issue	Reason	Fix
Job not running	timezone mismatch	set timeZone
Too many Jobs	no history limit	set limits
Overlapping Jobs	default policy	set Forbid
Missed executions	controller down	increase startingDeadlineSeconds

🔹 **CronJob vs Linux Cron (INTERVIEW)**

Feature	K8s CronJob	Linux Cron
HA	✅	❌
Retry	✅	❌
Observability	✅	❌
Scaling	✅	❌

🔹 **CronJob vs Job vs Deployment**

Feature	CronJob	Job	Deployment
Scheduled	✅	❌	❌
One-time	❌	✅	❌
Long-running	❌	❌	✅

Kubernetes CronJobs schedule Jobs using cron expressions, providing native retry, concurrency control, cleanup, and observability for periodic batch workloads.